

0.80.1. x et y rangés ensemble : struct — la boucle dans un fichier

Le 0.80, avec l'intérieur du `while` dans un autre fichier, `boucle.h`, versé au milieu de la boucle par `#include`.

1. Ce qu'il faut comprendre

C'est le 0.80, avec un seul changement : les trois lignes qui étaient **dans** le `while` (`image`, `deplace_croix`, les `nombre`) sont maintenant dans un **second fichier**, `boucle.h` : l'onglet à côté de `principal.cpp`.

Ce qui est nouveau ici : un `#include` **au MILIEU du programme**. Au 0.67, il était tout en haut. Mais `#include "boucle.h"` veut seulement dire « **verse ici** le contenu de ce fichier ». On peut donc le mettre **entre les accolades du** `while` : avant la compilation, la ligne est remplacée par les trois lignes de `boucle.h`, exactement là.

Le programme compilé est le même qu'au 0.80, au mot près : il est seulement rangé en deux.

`#include` **doit être seul sur sa ligne**, sans rien avant ni après : c'est pour cela qu'il est collé au bord gauche, et que son commentaire est au-dessus.

C'est possible, mais ce n'est pas la meilleure façon : en lisant `boucle.h` seul, on ne sait pas qu'il est versé dans une boucle, ni d'où vient `joueur`. Le 0.80.2 montre la façon propre, avec une fonction.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1775 octets** de cartouche.

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// Le nouveau type : la colonne et la ligne sous un seul nom (le 0.80).
```

2. UN ENREGISTREMENT — POSITION

```
struct Position { uint8_t x, y; };
```

3. UNE VARIABLE — JOUEUR

```
Position joueur;      // joueur.x et joueur.y
```

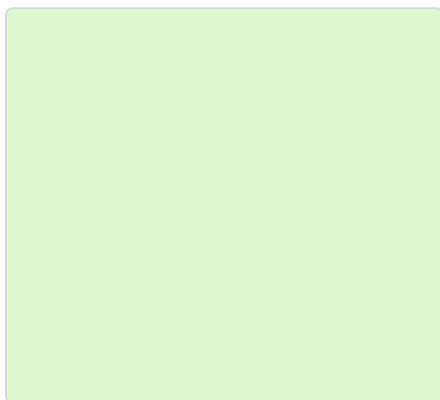
4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    joueur.x = 10;      // départ : (10, 0)
    joueur.y = 0;

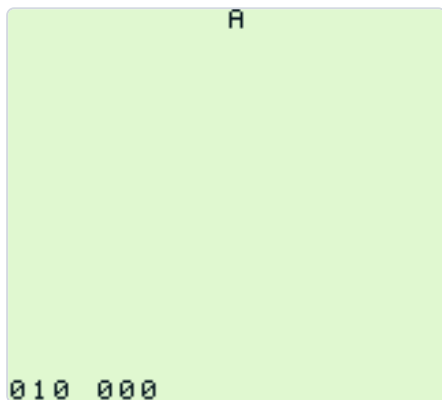
    while (true) {
        // Le changement : l'intérieur de la boucle est dans l'onglet boucle.h.
        // #include le verse ICI, à chaque tour, comme si ses lignes étaient
        // écrites entre ces accolades. (#include veut être seul sur sa ligne.)
#include "boucle.h"
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Exactement le 0.80 : un A qui suit la croix, sa position en bas. Mais la boucle est dans l'onglet boucle.h.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A part de (10, 0)
- ✓ la croix le déplace, depuis l'autre fichier (6, 2)
- ✓ il est à la place que disent joueur.x et joueur.y

Ensuite : **0.80.2. x et y rangés ensemble : struct — une fonction dans un fichier** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++